

Lab 4 Report

Authors:

Aidan Frondozo, 2274165

Kylie Neal, 2142029

Date: November 19, 2025

EE 474, Autumn, 2025

Introduction	3
System Design and Implementation	3
Code Structure	3
Discussion	4
Division of Work	4
Suggestions	4
Conclusions	5
References	5
Appendices	6
Total Number of hours Spent	6

Introduction

The main purpose of this lab was to gain familiarity with exploring preemptive algorithms and utilising the dual-core architecture of the ESP32 to introduce parallel computing. In this lab, we used the FreeRTOS platform for task scheduling and binary semaphores to implement a shortest time remaining scheduling algorithm and a light level anomaly detection system.

System Design and Implementation

In task 1, we used FreeRTOS task handles and global timing variables to manage the running of 3 separate tasks (blinking an LED, printing 1-20 to the LCD, and printing the alphabet to the Serial Monitor). Additionally, there was an additional task created to manage the scheduling. This task checks which task should be run next for the Shortest Time Remaining First scheduling algorithm, making sure to allow the longest task to run once the shortest tasks have completed.

In task 2, we used FreeRTOS's binary semaphores to synchronise data taken from our photoresistor to our display and LED alarm. We also utilised different cores to run our light-level anomaly detection system and our prime number calculation task in parallel.

Code Structure

In part 1 of the lab, there are global constant variables used to set the periods of the three executable tasks, volatile variables to update the current time remaining of the tasks, and handles to manage the running of the tasks. Then there are three task functions that describe the executable tasks. If there is still time remaining, it performs a small chunk of the function, deletes the respective time from the time remaining in the task, and then suspends the task. Next, there is a task without a handler, the scheduler task. This grabs the current remaining time on each task, resets the remaining time and peripherals if the task is ready to be reset, and then determines which task to run based on which has the shortest remaining time. The peripherals are all initialized and tasks are all created in the setup, and each of the execution tasks are initialized with a handle and suspended in setup. The loop function remains empty, as the program execution is driven by the FreeRTOS scheduler tasks.

In task 2 of the lab, it initializes the LCD, photoresistor, LED, serial interface, and a binary semaphore used for synchronizing tasks. A circular buffer stores the five most recent sensor readings, allowing the system to compute a 5-sample Simple Moving Average (SMA). Four FreeRTOS tasks run in parallel across the two CPU cores. The *lightDetector* task (core 0) samples the photoresistor every 500 ms, updates the buffer, computes the SMA, and signals other tasks whenever new data is ready. The *LcdDisplay* task (core 0) waits on this semaphore and updates the LCD only when the displayed values need refreshing. The *ledAlarm* task (core 1) checks each new reading and triggers a blinking LED pattern whenever the light level is abnormally high or low. In parallel, the *primeNumberCalculator* task (core 1) continuously

computes and prints prime numbers up to 50,000 to demonstrate multitasking and core utilization. All tasks are created and pinned to cores in `setup()`, while the `loop()` function remains empty because execution is fully driven by the FreeRTOS scheduler.

Discussion

In implementing part 1, there were multiple “bumps in the road.” The first step was realizing that the tasks should only perform a small “bit” of the entire task (i.e. print only one letter to the serial monitor) rather than complete the whole task. Once that was sorted out, it ran into the issue that it would only run the LED task because that task always had the shortest time remaining. The original solution to that was to only reset the tasks’ remaining times, but that turned the STRF scheduler into a Round Robin scheduler. The proposed solution was to only restart the shorter tasks once the longest task had been started; this way, each task would run, and the task that had the shortest remaining time at a given moment would be the task chosen to run its next “bit” of the task.

For task 2, the implementation was pretty straightforward. We followed the lab instructions and went step by step, starting by implementing each task individually and then initializing everything in the `setup()` function. With FreeRTOS, all the tasks are taken care of automatically, making it very easy for us to implement functions.

Division of Work

Kylie was responsible for completing task 1. Aidan was responsible for completing task 2. Both members contributed 50% of the work for this lab.

Suggestions

Our biggest complaint for this lab was how vague and ambiguous the spec was for task 1. Given the current lab document, it is very easy to implement a round-robin or preemptive scheduler instead of the intended shortest time remaining scheduler. We suggest that the task 1 description be made more clear and provide an example of the intended behaviour.

Conclusions

In conclusion, this lab successfully demonstrated how preemptive scheduling and dual-core execution can be leveraged on the ESP32 to manage concurrent tasks efficiently. By implementing both the shortest-time-remaining scheduler and the light-level anomaly detection system, we gained practical experience with FreeRTOS, binary semaphores, and real parallel processing.

References

Arduino. "Timer Interrupts on Arduino." Arduino Official Documentation, 2022.
<https://www.arduino.cc>.

Appendices

Total Number of hours Spent

Aidan: 4 hours

Kylie: 4 hours